

[HPC](#)[Running jobs \(Slurm\)](#)[Bases of Slurm](#)

Bases of Slurm

Helios cluster uses **Slurm** (Simple Linux Utility for Resource Management) for cluster/resource management and job scheduling.

Slurm is responsible for allocating resources to users, providing a framework for starting, executing and monitoring work on allocated resources and scheduling work for future execution.

! SLURM TERMINOLOGY

Slurm **job** consists in one or more **steps**, each consisting in one or more **tasks** each using one or more **CPU**.

- Jobs are typically created with the `sbatch` command
- Steps are created with the `srun` command
- Tasks are requested on the job level with `--ntasks` (`-n`) or `--ntasks-per-node`
- CPUs are requested per task with `--cpus-per-task` or `-c`.

Slurm main commands

Command	Description
<code>sbatch job.slurm</code>	Submit batch job
<code>srun <command></code>	Run step or program inside job
<code>scontrol show job <jobid></code>	Show information of a running jobid
<code>sacct -j <jobid></code> <code><options></code>	Show information of a completed jobid

Command	Description
<code>seff <jobid></code>	A tool to display the efficiency (CPU, MEM) of completed jobs
<code>scancel <jobid></code>	Cancel/kill jobs
<code>squeue</code>	Show running/pending jobs
<code>sinfo</code>	Show cluster informations

For more information about Slurm: <https://slurm.schedmd.com/quickstart.html>

Slurm configuration

In Slurm, multiple nodes can be grouped into **partitions** which are sets of nodes aggregated by shared characteristics or objectives, with associated limits for wall-clock time, job size, etc.

Partition	Description	#Nodes (cores)	#GPU (type)	TimeLimit (default)	Limits
mpi	MPI based applications	16 (384)	-	8-00:00:00 (24:00:00)	48 CPU per user
smp	SMP and array tasks based applications	5 (176)	-	8-00:00:00 (24:00:00)	32 CPU per user
gpu	Deep and machine learning applications	5 (320)	3x5 (A100)	12-00:00:00 (4:00:00)	1 GPU per user

Partition	Description	#Nodes (cores)	#GPU (type)	TimeLimit (default)	Limits
gpu_l40	Deep and machine learning applications	1 (128)	2x1 (L40)	12-00:00:00 (4:00:00)	1 GPU per user
visu	HPC remote visualization	1 (64)	3x1 (A40)	8-00:00:00 (4:00:00)	1 GPU per user
bigmem	Memory-intensive applications	2 (64+48) 256+1024 GB memory	-	8-00:00:00 (4:00:00)	32 CPU per user

Resources limitations are enforced using **QoS**. Each QoS is attached to a user or account.

We can view available QoS:

```
$ sacctmgr show qos format=Name,Priority,MaxTRESPerUser%20
      Name      Priority      MaxTRESPU
-----
normal          0      cpu=48,gres/gpu=1
meso             0      gres/gpu=3
2gpu             0      gres/gpu=2
cpu-72           0      cpu=72
cpu-96           0      cpu=96
cpu-64           0      cpu=64
3gpu             0      gres/gpu=3
```

Check if a QoS is attached to a user, for example user `demo_meso`:

```
$ sacctmgr show associations user=demo_meso format=Account,User,QOS
Account      User      QOS
-----
meso  demo_meso  2gpu,cpu-64,cpu-72,+
```

If a user have QoS attached, he needs to request it, for example:

```
$ sbatch --qos=cpu-64 script.slurm
```

**TIP**

Use `sinfo` or `scontrol show partitions` commands to show more information about partitions.

```
$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
mpi        up 8-00:00:00    10 drain* node4-[7-8,13-20]
mpi        up 8-00:00:00     1 alloc node4-2
mpi        up 8-00:00:00     5 idle node4-[1,3-6]
smp        up 8-00:00:00     1 drain* node4-12
smp        up 8-00:00:00     4 idle node4-[9-11,26]
gpu        up 12-00:00:0     3 drain* node4-[23-25]
gpu        up 12-00:00:0     1 mix node4-21
gpu        up 12-00:00:0     1 idle node4-22
gpu_l40    up 12-00:00:0     1 idle node4-28
visu       up 8-00:00:00     1 idle m3sovisu
bigmem     up 8-00:00:00     2 idle m3sovisu,node4-27
```

GPU and GPU_L40 partitions

GPU partition settings

This partition provides 5 AMD (Zen2) nodes, each have the following settings:

- 252 GB of memory
- 32 cores
- 3x Nvidia A100

GPU_L40 partition settings

This partition provides 1 AMD (Zen4) node, which have the following settings:

- 1024 GB of memory
- 128 cores
- 2x Nvidia L40

Common settings

Slurm partition settings:

- Limit 1 GPU per user
- Limit 128 GB memory per job
- 16 GB default memory allowed per CPU (use `--mem=72G` for example to request more)
- Partition TimeLimit is 8 days (`8-00:00:00`)
- Default job WallTime is 8 hours (`--time 8:00:00`) if is not given (use `--time` or `-t` option to override this value)

! SCRATCH DIRECTORY

- Each node in this partition provides a local scratch directory based on SSD. It's suitable for intensive IO applications.
- You can access to your folder using `$GPU_SCRATCH_DIR` environment variable inside your Slurm job
- This folder is **purged** when needed.

Interactive mode

Objective: request GPU resource and open an interactive shell.

Example:

```
$ srun --partition=gpu --time=4:00:00 --gres=gpu:1 --pty bash -l
```

Where:

- `--partition` use gpu partition

- `--time` request 4h (format HH:MM:SS)
- `--gres=gpu:1` request 1 GPU on the node
- `--pty bash -l` open bash session

MEMORY

Please note that default memory used is 16GB.

You can request more by using `--mem` option. (example: `--mem=72G`)

Once connected, you can use `$WORK` or `$GPU_SCRATCH_DIR` folders as working space.

A typical workflow on GPU node looks like this:

```
$ cd $WORK
$ module load miniconda3@4.10.3
$ conda env list
$ conda create -n meso_test # optional
$ conda activate meso_test
(meso_test) $ conda list
```

Install pandas for example:

```
(meso_test) $ conda install pandas
```

Batch mode

Here is an example of Slurm script to use:

gpu.slurm

```
#!/bin/bash -l

## Slurm job script to be adapted as required ##

#SBATCH --job-name="MY_JOB_NAME" ## <---(modify me)
#SBATCH --output=%x.%J.out        ## %x=job name, %J=job id
```

```
#SBATCH --error=%x.%J.out
#SBATCH --time=24:00:00      ## 24h walltime ((hh:mm:ss) <--- (modify me))
#SBATCH --partition=gpu      ## gpu partition
#SBATCH --gres=gpu:1

#SBATCH --mem=10G           ## 10G per node <---(modify me)
                           ## Please don't use more than 128g.
#SBATCH --mail-user=<yourmail>@.yourdomain.fr  ## <---(modify me)
#SBATCH --mail-type=END,FAIL  ## notify when job end/fail

## view allocated GPU cards
nvidia-smi

module purge
module load miniconda3@4.10.3/gcc-12.1.0

conda activate your_env && python GPU_program.py
```

Submit job to Slurm:

```
$ sbatch gpu.slurm
```

Check job status:

```
$ squeue
```

MPI partition

This partition is used for **parallel MPI** applications.

- Number of MPI tasks (`-n` option) must be divisible by 24. For example 24, 48, ...
- Default memory per core/slot is 4GB
- Don't use `mpirun` command to execute application, use `srun` command instead
- By default, you can use up to 48 MPI tasks:
 - If you need up to 72 tasks add `#SBATCH --qos=cpu-72` before your `#SBATCH -n` line,

- if you need more than 72 tasks, please ask the administrators

WARNING

For MPI applications, do not use `--node` or `(-N)` Slurm option.

Use `--ntasks` or `(-n)` instead to request tasks from nodes.

Here is an example of MPI job to be adapted to your needs:

```
mpi.slurm
```

```
#!/bin/bash -l
# Fichier submission.SBATCH

#SBATCH --job-name="MON_JOB"
#SBATCH --output=%x.%J.out    ## %x=job name, %J=job id
#SBATCH --error=%x.%J.out
# walltime (hh:mm:ss) max is 8 days
#SBATCH --time=24:00:00
#SBATCH --partition=mpi
#SBATCH --ntasks=48    ## request 48 MPI tasks
#SBATCH --mem=0        ## request all node's memory
#SBATCH --mail-user=votreadresseufc@univ-fcomte.fr ### PLEASE EDIT ME
#SBATCH --mail-type=END,FAIL    # notify when job end/fail

module purge
module load openmpi@2.1.6/gcc-12.1.0

srun ./myMPI_Application listParams
```

Submit your job to Slurm:

```
$ sbatch mpi.slurm
```

SMP partition

This partition is used for SMP (like OpenMP and multithreaded based applications) and Array

tasks applications.

- 4x nodes (32 cores and 96 GB memory)
- Default memory per core/slot is 4 GB

WARNING

For Multithreaded, OpenMP application, do not use `-N` nor `-n` Slurm option.
Use `--cpus-per-task` or `-c` instead to request cores from a single node.

Interactive mode

Objective: request resources and open an interactive shell.

Example:

```
$ srun --partition=smp --time=4:00:00 --cpus-per-task=8 --mem=32G --pty bash -l
```

Where:

- `--partition` use smp partition
- `--time` request 4h (format HH:MM:SS)
- `--cpus-per-task` request 8 cores
- `--mem=32G` request 32 GB memory
- `--pty bash -l` open bash session

Batch mode

Multithreaded applications

Objective: request 1 task with multiple cores within a node.

- Use `-c` or `--cpus-per-task` to request cores. Maximum value is 32, hardware limit.
- Use `--mem` to request memory per node.

Here is an example of OpenMP job to be adapted to your needs:

```
openmp.slurm
```

```
#!/bin/bash -l
# Fichier submission.SBATCH

#SBATCH --job-name="MON_JOB"
#SBATCH --output=%x.%J.out    ## %x=job name, %J=job id
#SBATCH --error=%x.%J.out
# walltime (hh:mm:ss) max is 8 days
#SBATCH --time=24:00:00
#SBATCH --partition=smp
#SBATCH --cpus-per-task=16    ## request 16 cores (MAX is 32)
#SBATCH --mem=16G    ## (MAX is 96G)
#SBATCH --mail-user=<votreadresseufc>@univ-fcomte.fr    ### PLEASE EDIT ME
#SBATCH --mail-type=END,FAIL    # notify when job end/fail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
srun ./my_OPENMP_Application listParams
```

Submit your job to Slurm:

```
$ sbatch openmp.slurm
```

Array tasks

Objective: Submit a number of identical jobs without having to drive the submission with an external script, use the Slurm's feature of array jobs.

- Use `--array x-y` parameter, where `x` and `y` are the array bounds.
- An Array Task is identified with `$SLURM_ARRAY_TASK_ID` environment variable.

Here is an example of array job to be adapted to your needs:

```
array.slurm
```

```
#!/bin/bash -l
```

```
# Fichier submission.SBATCH

#SBATCH --job-name="MON_JOB"
# walltime (hh:mm:ss) max is 8 days
#SBATCH --time=24:00:00
#SBATCH --partition=smp
#SBATCH --output=array_%A-%a.out    # Standard output and error log
#SBATCH --array=1-5                 # Array range
#SBATCH --mem=8G
#SBATCH --mail-user=<votreadresseufc>@univ-fcomte.fr ### PLEASE EDIT ME
#SBATCH --mail-type=END,FAIL        # notify when job end/fail

echo $SLURM_ARRAY_TASK_ID
srun ./myarray input_${SLURM_ARRAY_TASK_ID} > output_${SLURM_ARRAY_TASK_ID}
```

Submit your job to Slurm:

```
$ sbatch array.slurm
```

Sequential applications

The script is trivial. By default, Slurm allows 1 core and 4 GB memory.

```
seq.slurm
```

Submit your job to Slurm:

```
$ sbatch seq.slurm
```

Bigmem partition

This partition is used for memory intensive application. We can use OpenMP like, multithreaded and array tasks applications.

- 1x node 48 cores and 1024 GB memory
- 1x node 64 cores and 256 GB memory

```
#!/bin/bash -l
# Fichier submission.SBATCH

#SBATCH --job-name="MON_JOB"
#SBATCH --output=%x.%J.out    ## %x=job name, %J=job id
#SBATCH --error=%x.%J.out
# walltime (hh:mm:ss) max is 8 days
#SBATCH --time=4:00:00
#-----

$ srun --partition=bigmem --time=4:00:00 --cpus-per-task=8 --mem=32G --pty
bash -l

#SBATCH --mail-type=END,FAIL    # notify when job end/fail

srun ./my_seq_application
```

- `--time` request 4h (format HH:MM:SS)
- `--cpus-per-task` request 8 cores
- `--mem` request 32 GB memory
- `--pty bash -l` open bash session

Batch mode

Here is an example of Slurm job to be adapted to your needs:

bigmem.slurm

```
#!/bin/bash -l
# Fichier submission.SBATCH

#SBATCH --job-name="MON_JOB"
#SBATCH --output=%x.%J.out    ## %x=job name, %J=job id
#SBATCH --error=%x.%J.out
# walltime (hh:mm:ss) max is 8 days
#SBATCH --time=24:00:00
#SBATCH --partition=bigmem
#SBATCH --cpus-per-task=16    ## request 16 cores (MAX is 32)
#SBATCH --mem=32G    ## (MAX is 250G)
#SBATCH --mail-user=<votreadresseufc>@univ-fcomte.fr    ### PLEASE EDIT ME
```

```
#SBATCH --mail-type=END,FAIL    # notify when job end/fail
```

```
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
```

```
srun ./my_OPENMP_Application listParams
```

Submit your job to Slurm:

```
$ sbatch bigmem.slurm
```